

Workload acceleration by optimizing materialized view selection using local search

Anonymous Author(s)

Paper under review

Confidential

Abstract

The growing size of database workloads has made view selection a key performance challenge. Materializing frequent sub-queries in workloads improves query efficiency, but it incurs significant view maintenance costs due to updates. Although existing methods such as BIGSUBS address this trade-off between the benefit of using materialized views and the overhead of view maintenance, they have two drawbacks: insufficient maintenance cost modeling and ineffective view selection due to probabilistic techniques. We propose a novel view selection method that incorporates incremental view maintenance cost directly into the optimization objective of an integer linear program and applies local search to efficiently explore the solution space. In order to apply local search to the view selection problem, we develop neighboring solutions using sub-query containment, and select initial solutions based on sub-query frequency, utility, or utility per storage unit. Experiments using Redbench, a benchmark simulating real-world query workloads on Amazon Redshift, show that our approach outperforms BIGSUBS in both optimization utility and the quality of selected views.

CCS Concepts

• Information systems;

Keywords

View selection, Materialized view, Integer linear programming

ACM Reference Format:

Anonymous Author(s). 2025. Workload acceleration by optimizing materialized view selection using local search. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM '25)*, November 10–14, 2025, Coex, Seoul, Korea. ACM, New York, NY, USA, 10 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

In recent years, the growing number of database users has significantly increased the volume of executed queries in workloads, making the optimization of workload performance a critical challenge. [10, 14, 16, 20]. A materialized view is a common database technique that stores the physically materialized result set of a query. Materializing the results of frequent sub-queries in workloads is

a simple but effective way to significantly improve the workload performance [11, 18]. However, when updates are made to the underlying tables, the corresponding materialized views must be maintained, which can be time-consuming. Therefore, there is a trade-off between the benefit of using materialized views and the overhead of view maintenance.

Motivation. Several materialized view selection methods have been proposed to balance the trade-off [14, 20, 22, 27, 28]. Specifically, BIGSUBS [14] is designed for large-scale workloads, which accelerates view selection by iteratively choosing 1) which sub-queries to materialize and 2) which sub-queries to use for evaluating the queries in the workload. In addition, BIGSUBS limits the total size of selected views by imposing a storage capacity constraint, expecting that it mitigates significant performance degradation during updates. However, there are two issues with BIGSUBS caused by its design, 1) the storage capacity constraint is not sufficiently effective to reflect the overhead of view maintenance, and 2) the quality of solutions found by BIGSUBS tends to be low because it relies on probabilistic techniques for view selection.

Contribution. To address the above two issues, we propose a novel method that balances the trade-off between the benefit of using materialized views and the overhead of view maintenance. Our method offers two key contributions. First, by incorporating incremental view maintenance costs into the objective function of an integer linear program, we ensure that the overhead of view maintenance is directly reflected in the optimization process. Second, we employ a well-known optimization heuristic, local search [1], which effectively explores the solution space to find high-quality approximations. In detail, applying local search to the materialized view selection poses two technical challenges: defining appropriate neighboring solutions and selecting suitable initial solutions. To define neighboring solutions, we leverage the insight that sub-queries with a containment relationship tend to provide similar benefits of using materialized views for workloads. We treat two sub-queries as neighbors if one contains the other or is contained by the other. For initial solution selection, we use simple three methods based on sub-query characteristics: high frequency, high utility, and high utility per storage unit in the workload.

We conduct experiments to evaluate our proposed method in terms of optimization speed of view selection, the quality of selected views, and the workload performance using Redbench, a recent benchmark that simulates query frequencies in real-world production queries in Amazon Redshift based on the Join Ordering Benchmark (JOB) [17]. These experiments demonstrate that our approach outperforms the state-of-the-art method, BIGSUBS.

Organization. The remainder of this paper is organized as follows. Section 2 describes preliminaries. Section 3 presents the overview

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CIKM '25, Coex, Seoul, Korea

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

of our workflow, and Sections 4 and 5 describe methods of view selection and query rewriting in detail, respectively. Section 6 presents experimental results to demonstrate its effectiveness. Section 7 discusses related work, and Section 8 concludes the paper with a summary and directions for future work.

2 Preliminaries

A workload consists of a set of queries, $Q = \{q_1, q_2, \dots, q_n\}$ where q_i and q_j for all i, j ($i \neq j$) are different and $|Q| = n$ denotes the number of queries in the workload. Let $S = \{s_1, s_2, \dots, s_m\}$ be a set of sub-queries extracted from Q where $|S| = m$ is the number of sub-queries. The set S of sub-queries is denoted as follows:

$$S = \bigcup_{q_i \in Q} \text{extractSubQueries}(q_i), \quad (1)$$

where the function `extractSubQueries()` takes a query as input and returns the set of sub-queries corresponding to partial execution plans extracted from the execution plan of q_i . Since execution plans are represented as tree structures, the resulting sub-queries may exhibit containment relationships, i.e., a sub-query may include another.

Figure 1 shows a workload including four queries q_1 to q_4 . Each query has a tree-structured execution plan. For example, q_2 includes six sub-queries. Totally, there are 9 sub-queries in the workload.

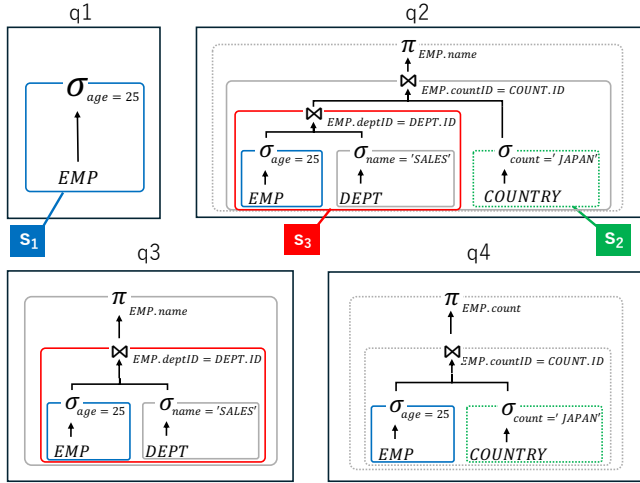


Figure 1: An example of workload. Sub-queries shared across multiple queries: sub-query s_1 (blue line) is shared by four queries q_1 - q_4 , sub-query s_2 (green line) is shared by q_2 and q_4 , and sub-query s_3 (red line) is shared by q_2 and q_3 .

2.1 Sub-query Utility and Storage Size

View utility [14] is a common notion that represents the effectiveness of a view for a given query and database. The utility of a sub-query is defined as follows:

DEFINITION 1. *The utility of sub-query s_j for query q_i in a database is defined as the reduction in the estimated execution cost of*

q_i ¹ *using the materialized result of s_j :*

$$u_{ij} = \text{cost}_{\text{read}}(q_i) - \text{cost}_{\text{read}}(q_i|s_j), \quad (2)$$

where $\text{cost}_{\text{read}}(q_i)$ and $\text{cost}_{\text{read}}(q_i|s_j)$ are the estimated costs of executing q_i without and with utilizing the materialized result of s_j , respectively.

The total utility U_j of sub-query s_j is defined as the sum of its utility for each query q_i across Q :

$$U_j = \sum_{q_i \in Q} u_{ij}. \quad (3)$$

We hereby note that if q_i does not include s_j , u_{ij} is zero.

We define that a storage size b_j of s_j is the estimated size of the result executed by s_j .

2.2 Problem Definition

We solve the following problem in this paper.

Problem definition. *Given a database, $Q = \{q_1, q_2, \dots, q_n\}$, and storage budget B_{max} , we aim to find a set of sub-queries $S \subseteq S$ as views that minimize workload execution time and view maintenance time where the total estimated size of views is less than B_{max} . We assume that update operations are not given.*

Materializing common sub-queries shared across multiple queries enables improving workload performance. As illustrated in Figure 1, sub-query s_1 (blue line) is shared by four queries q_1 - q_4 , and sub-query s_2 (green line) is shared by q_2 and q_4 . However, in addition to the benefits of reusing materialized views, we have an additional cost of view maintenance when the related base tables are updated.

This observation naturally raises a fundamental question: which sub-queries should be materialized to improve the workload performance? While s_1 appears in four queries, its relatively small query plan suggests limited benefit from materialization. Conversely, s_3 includes an expensive join operation, which may be effective for reuse; however, it may incur higher view maintenance costs. These trade-offs reveal the complexity of the materialized view selection problem.

3 Overview of Our Workflow for Selecting and Using Views

This section presents an overview of our workflow for selecting and using views to achieve a balance between the benefit of using materialized views and the overhead of view maintenance. Figure 2 provides a workflow, which consists of 1) sub-query extraction, 2) materialized view selection, 3) view materialization, and 4) rewriting queries using materialized views.

The materialized view selection offers several key contributions. First, by incorporating incremental view maintenance costs into the objective function, we ensure that the overhead of view maintenance is directly reflected in the optimization process (Section 4.1). Second, we employ a well-known optimization heuristic, local search [1], which effectively explores the solution space to find high-quality approximations (Section 4.2). Third, we define the incremental view maintenance cost of materialized views in response to update operations in the workload (Section 4.3), and then we describe the

¹We can employ `EXPLAIN` command for the query execution cost provided by the relational database system.

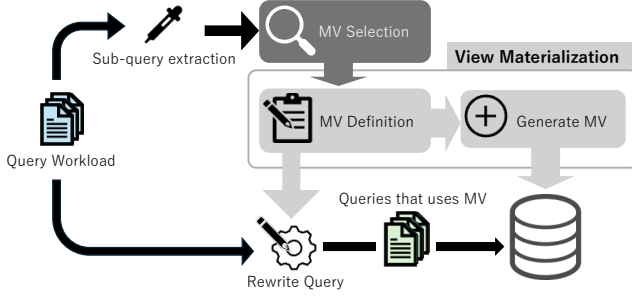


Figure 2: Workflow consisting of 1) sub-query extraction, 2) materialized view (MV) selection, 3) view materialization, and 4) rewriting queries using materialized views.

algorithm of the materialized view selection (Section 4.4). Finally, we generate materialized views and rewrite the original query to use the views (Section 5).

4 Method for View Selection

We describe how to select views in detail.

4.1 Objective Function for View Selection

The objective of materialized view selection is to balance the trade-off between the benefit of using materialized views and the overhead of view maintenance. This problem can be formulated as an integer linear program (ILP) that maximizes the difference between the utility of sub-queries and the view maintenance cost.

DEFINITION 2 (VIEW SELECTION PROBLEM). Let u_{ij} be the sub-query utility as defined in Equation (2), m_j be the incremental maintenance cost of materialized sub-query s_j^2 , b_j be the storage size of s_j , and B_{max} be the storage budget. Given queries Q and sub-queries S , the view selection problem is defined as follows:

$$L(Q, S) = \text{maximize} \sum_{q_i \in Q} \sum_{s_j \in S} u_{ij} \cdot y_{ij} - \sum_{s_j \in S} m_j \cdot z_j$$

$$\text{subject to} \sum_{s_j \in S} b_j \cdot z_j \leq B_{max} \quad (4)$$

$$y_{ik} + \frac{1}{|S|} \sum_{j \neq k} y_{ij} \cdot x_{jk} \leq 1 \quad \forall i \in [1, |Q|], \forall k \in [1, |S|] \quad (5)$$

$$y_{ij} \leq z_j \quad \forall i \in [1, |Q|], \forall j \in [1, |S|] \quad (6)$$

$L(Q, S)$ is the objective function, which maximizes the sum of the utility of sub-queries over all queries (first term) minus the incremental view maintenance cost (second term). The utility of sub-queries is computed as $u_{ij} \cdot y_{ij}$, where the decision variable y_{ij} indicates whether query q_i uses the materialized view of s_j . The view maintenance cost is computed as $m_j \cdot z_j$, where the decision variable z_j indicates whether s_j is selected for materialization. Equations (4)–(6) define the constraints of the problem:

- The storage constraint (4) ensures the total size of materialized views does not exceed B_{max} .

- The containment constraint (5) prevents overlapping views from being used in the same query. Binary variable x_{jk} indicates whether s_j contains s_k .
- The materialization constraint (6) ensures that only materialized sub-queries can be used for query processing.

Solving this integer programming formulation yields: 1) which sub-queries s_j should be materialized ($z_j = 1$), and 2) which query q_i should use which sub-query s_j ($y_{ij} = 1$).

4.2 Optimization using Local Search

Naively solving the optimization formulation presented in Section 4.1 as an Integer Linear Programming (ILP) leads to severe performance degradation as the number of queries increases. Indeed, ILP is known to be NP-hard and typically incurs an exponential time cost with respect to the number of decision variables, i.e., y_{ij} and z_j where the size depends on the product of query size ($|Q|$) and sub-query size ($|S|$). Furthermore, although BIGSUBS [14] is designed for large-scale workloads and accelerates view selection, the quality of its solutions tends to be low due to its reliance on probabilistic techniques for view selection.

To address this, we employ a well-known optimization heuristic, local search [1], to iteratively approximate the optimal materialized views. Local search is a technique that avoids exhaustive solution enumeration by incrementally exploring the solution space through the enumeration of neighboring solutions to the current one, thereby enabling iterative refinement.

In order to apply the local search to the materialized view selection problem, we first generate promising initial candidate sub-queries (Section 4.2.1). Then, we explore neighboring solutions by modifying the current candidate sub-queries to explore the solution space. For the expanded solutions by adding neighboring solutions, we solve an ILP to select new candidates that maximize the objective function (Section 4.2.2). This process is repeated until no further improvement can be achieved in the objective function (Section 4.2.3).

4.2.1 Initial Solution Selection. We propose three methods for selecting promising initial candidate sub-queries. We use one of the following metrics for sub-query s_j : 1) frequency F_j in the workload, 2) total utility U_j , or 3) utility per unit storage $E_j = U_j/b_j$. Based on the selected metric, the top k sub-queries are chosen as the initial candidate sub-queries, such that the total storage cost does not exceed the storage budget B_{max} .

topk-F: This method simply prioritizes sub-queries that appear most frequently in the workload. By favoring commonly shared sub-queries, it may enhance the effectiveness of selecting initial views.

topk-U: This method selects sub-queries with the highest total utility, as defined in Equation (3). It is expected to be more effective than topk-F because it explicitly accounts for the actual benefit of utilizing materialized views. While this approach successfully identifies high-utility candidates, it may also select sub-queries that require substantial storage, potentially reducing storage efficiency.

topk-E: This method balances utility and storage requirements by selecting sub-queries based on their utility per unit of storage.

4.2.2 Neighborhood-based sub-query exploration. To explore the solution space via local search, we first define the neighbors of sub-queries. Our approach leverages containment relationships

²Section 4.3 describes the detail of incremental view maintenance cost m_j .

in query plan trees: if the plan tree of one sub-query contains that of another, these two sub-queries are considered neighbors. A parent sub-query expresses a larger plan tree that fully contains those of its child sub-queries, thereby it materializes a greater portion of the overall query processing. In contrast, child sub-queries are more likely to be shared across multiple queries due to their smaller structure of plan trees.

We define the neighbors of sub-queries as follows.

DEFINITION 3 (NEIGHBORS OF SUB-QUERY). *The neighbors of a sub-query s consist of s itself, its parent sub-queries, and its child sub-queries:*

$$\text{neighbors}(s) = \text{parent}(s) \cup \text{children}(s) \quad (7)$$

where $\text{parent}(s)$ denotes the set of sub-queries whose root nodes are parents of the root node of sub-query s , while $\text{children}(s)$ denotes the set of sub-queries whose root nodes are children of the root node of sub-query s .

In Figure 1, parents and children of s_3 are indicated by solid-lined rectangles. The parents are sub-queries indicated by rectangles that cover red rectangles in q_2 and q_3 , and the children are sub-queries indicated by sub-queries within the red rectangle, respectively.

The neighbor-based sub-query exploration enumerates promising sub-tree candidates that balance the greater utility of materialization (achieved by enumerating parent sub-queries) and the higher potential for reuse across multiple queries (achieved by enumerating child sub-queries).

4.2.3 Iterative Optimization. Starting from the initial candidate sub-queries, we explore the solution space by adding neighbor sub-queries and then choose new candidate sub-queries that maximize the objective function. Let C_t denote the candidate sub-queries at iteration t ($t > 0$), and let E_t denote the extended sub-queries obtained by adding neighbor sub-queries to C_t , defined as follows:

$$E_t = \bigcup_{s \in C_t} s \cup \text{neighbors}(s) \quad (8)$$

where C_0 is the initial solutions selected by the methods described in Section 4.2.1. Then, the new candidate sub-queries C_{t+1} are computed by solving an ILP over the extended sub-queries E_t :

$$C_{t+1} = \text{ILP}(Q, E_t) \quad (9)$$

The function $\text{ILP}(Q, E_t)$ selects the solutions to the view selection problem when the whole candidate sub-query set S is localized to a subset E_t , which corresponds to $L(Q, E_t)$ in Definition 2. This localization is a key advantage of the local search approach, as it significantly reduces the number of decision variables (y_{ij} and z_j) compared to the whole sub-query set S , that is $|E_t| \ll |S|$. The iteration continues until no further improvement can be achieved in the objective function.

4.3 Incremental View Maintenance Cost

This section describes the details of the incremental view maintenance cost m_j for a materialized view of subquery s_j in response to update operations. Since the workload contains update operations to the base tables, the materialized views must be maintained to reflect these changes. Numerous studies [2, 3, 7, 24] have explored

incremental view maintenance, and some of these techniques have even been integrated into commercial systems, such as Oracle, SQL Server, and Tempura [24]. Therefore, we focus on computing the cost of incremental view maintenance rather than that of full refresh-based view maintenance.

4.3.1 Notations. The incremental view maintenance cost in response to updates on base tables is expressed using a cost function $\text{cost}_{\text{IVM}}(\Delta)$, where the argument Δ denotes the delta view expression for given update operations on base tables. Δ is expressed as an algebraic expression of the view, in which each base table is updated with its corresponding delta table that reflects incremental update operations. As an example of a single-table view, Δ is expressed using a delta table, such as ΔA for Table A , and the incremental view maintenance cost is denoted as $\text{cost}_{\text{IVM}}(\Delta A)$. If a view is defined as a multi-table view, for example, for a given view $A \bowtie B$ and an incremental update operation ΔB to the base table B , the incremental view maintenance cost is denoted as $\text{cost}_{\text{IVM}}(A \bowtie \Delta B)$.

4.3.2 Cost Estimation for Incremental Join View Maintenance. In order to solve the view selection problem, we need to estimate the view maintenance cost (m_j) in Definition 2. However, since the database stores only the base tables and does not materialize the views in advance, the view maintenance cost must be estimated without physically generating the views. We begin by decomposing the incremental view maintenance into three constituent operations, and then describe how to estimate their costs.

Decomposing incremental view maintenance cost. First, the incremental view maintenance cost is calculated as the sum of the costs of three operations³: 1) computing changes to the view, 2) identifying records to update in the view, and 3) applying the changes to the view. Since the view maintenance cost of a single-table view is intuitive, we describe the incremental view maintenance cost by focusing specifically on binary join views. This approach can be easily extended to views defined by multi-table joins, as they can be represented as sequences of binary join operations in the execution plan.

Consider a binary join view, $V = A \bowtie B$, and an incremental delete operation $-\Delta B$ on Table B . We omit the cases for incremental insert and update operations, as they can be straightforwardly derived from the delete case. Let $\Delta V = A \bowtie \Delta B$. The incremental view maintenance cost is decomposed into the costs of the three operations and is expressed as follows.

$$\text{cost}_{\text{IVM}}(A \bowtie \Delta B) = \text{cost}(A \bowtie \Delta B) + \text{cost}(\sigma_{\Delta V}(V)) + \text{cost}(V - \Delta V) \quad (10)$$

Estimating incremental view maintenance cost: Then, we estimate the cost of the above three operations as follows.

1) Computing changes to the view: $\text{cost}(A \bowtie \Delta B)$ is approximated as the cost of querying $A \bowtie B$ times $\frac{\Delta B}{|B|}$ (where $|B|$ is the number of records in B), because we only need to compute ratio $\frac{\Delta B}{|B|}$ of $A \bowtie B$.

$$\text{cost}(A \bowtie \Delta B) \simeq \text{cost}_{\text{read}}(A \bowtie B) \times \frac{\Delta B}{|B|} \quad (11)$$

³If the update operations on the base table are insertion, the maintenance cost consists only of 1) and 3), because we can skip 2).

Algorithm 1 View selection optimization

Input: Q (queries), S (sub-queries), $U, \mathbf{b}, \mathbf{m}, X$, and $B_{\max}$ (storage budget)
Output: z, Y

```

1:  $z = \text{initial\_solution}_{B_{\max}}(S, U, \mathbf{b})$  // initial solution selection
2:  $U_{\text{previous}} = 0$ 
3: while  $\text{true}$  do
4:    $C = \text{candidate\_views}(z)$  //  $z$  to  $C$  transformation
5:    $E = \bigcup_{s \in C} s \cup \text{neighbors}(s)$  // sub-query exploration
6:    $z, Y = \text{ILP}_{B_{\max}, X, \mathbf{m}, U}(Q, E)$  // applying ILP
7:    $U_{\text{current}} = \sum_{q_i \in Q} \sum_{s_j \in S} u_{ij} \cdot y_{ij} - \sum_{s_j \in S} m_j \cdot z_j$ 
8:   // iteration termination check
9:   if  $U_{\text{previous}} \geq U_{\text{current}}$  then
10:    break
11:  end if
12:  // preparing for the next iteration
13:   $U_{\text{previous}} = U_{\text{current}}$ 
14: end while
15: return  $z, Y$ 

```

2) Identifying records to update in the view: The cost of identifying ΔV in the materialized view V is approximated as the cost of evaluating $\sigma_{\Delta B}(B)$ times fanout $_{A \bowtie B}(B \rightarrow V)$, where fanout $_{A \bowtie B}(B \rightarrow V)$ denotes the fanout (i.e., output cardinality) from B to V when V is obtained by $A \bowtie B$.

$$\text{cost}(\sigma_{\Delta V}(V)) \approx \text{cost}_{\text{read}}(\sigma_{\Delta B}(B)) \times \text{fanout}_{A \bowtie B}(B \rightarrow V) \quad (12)$$

For example, if tables A and B are in a one-to-many relationship (i.e., one A record corresponds to multiple B records) and the view is defined using natural join, then fanout $_{A \bowtie B}(B \rightarrow V) \approx 1$. Conversely, if they are in a many-to-one relationship, then fanout $_{A \bowtie B}(B \rightarrow V) \approx |A|/|B|$. If the view is defined using ad-hoc join, we use a simple histogram-based approach commonly used in relational databases. It is our future work to employ machine-learning-based cardinality techniques [25, 26] for more precise fanout estimation.

3) Applying the changes to the view: The cost of applying the changes ($-\Delta V$) to V , $\text{cost}(V - \Delta V)$, is approximated as the cost of deleting ΔB from the base table B times fanout $_{A \bowtie B}(B \rightarrow V)$.

$$\text{cost}(V - \Delta V) \approx \text{cost}_{\text{write}}(-\Delta B) \times \text{fanout}_{A \bowtie B}(B \rightarrow V) \quad (13)$$

where $\text{cost}_{\text{write}}(-\Delta B)$ denotes the cost of deleting ΔB from its base table.

4.4 Algorithm Description

We define U (sub-query utilities), $\mathbf{b}$ (sub-query footprints), $\mathbf{m}$ (sub-query maintenance costs), and z (sub-query for materialization) as vectors consisting of elements U_j, b_j, m_j , and z_j , respectively, for sub-query s_j ($1 \leq j \leq |S|$). We also define $X \in \{0, 1\}^{|S| \times |S|}$ as a sub-query containment matrix consisting of elements x_{jk} , which indicates whether s_j contains s_k , and $Y \in \{0, 1\}^{|Q| \times |S|}$ as a matrix consisting of elements y_{ij} , which indicates whether query q_i uses the materialized view of sub-query s_j .

The view selection algorithm is outlined in Algorithm 1. Initially, an initial candidate sub-queries (expressed using z) is generated

using one of the metrics topk- F , topk- U , or topk- E , subject to the storage budget (Line 1-2) as detailed in Section 4.2.1. Then, we conduct a local search and ILP until no further improvement can be achieved (Lines 3-14). Line 5 corresponds to neighbor-based sub-query exploration (detailed in Section 4.2.2), and Line 6 involves applying Integer Linear Programming (ILP) optimization for the current candidate sub-queries (E). ILP updates the decision variables (z, Y) that maximize the objective function in Definition 2. Lines 9-11 determine termination based on convergence criteria, and Line 15 outputs the final decision variables (z, Y).

5 View Materialization and Query Rewriting

Once the optimization of view selection is completed, we generate the materialized views and rewrite the original queries using the views based on the decision variables (z, Y) where y_{ij} indicates whether query q_i uses the materialized view s_j , and z_j indicates whether s_j is selected for materialization.

View generation. To define the materialized view, we extract all referenced tables, join conditions, and selection conditions from the execution plan of each sub-query s_j and construct a CREATE MATERIALIZED VIEW statement.

- **CREATE MATERIALIZED VIEW** as [view_name]
- **SELECT clause:** All columns from the referenced tables are included. The duplicate column names between different tables are resolved via renaming.
- **FROM clause:** All tables from the referenced tables.
- **WHERE clause:** All conditions from the referenced tables.

When materialized views are defined, they are subsequently materialized within the database.

Query rewrite using views. Each query is rewritten as follows based on y_{ij} : query q_i uses the materialized view s_j :

- **SELECT clause:** Replace the columns of the base tables in q_i with the corresponding columns in s_j .
- **FROM clause:** Replace the corresponding base tables in q_i with s_j .
- **WHERE clause:** Remove the join and filter conditions in q_i if they are included in s_j .

6 Experiments

The purpose of the experiments is to compare the performance of the proposed method with those of the baselines, a naive ILP-based method (Naive ILP) and BIGSUBS [14]. The evaluation metrics include workload execution time, optimization time for materialized view selection, total view utility, and view materialization time. We use the EXPLAIN command in PostgreSQL for estimating the utility: sub-query execution cost ($\text{cost}_{\text{read}}(\cdot)$) and the update operation cost ($\text{cost}_{\text{write}}(\cdot)$).

In detail, we conducted four experiments based on the following objectives:

Experiment 1: Workload performance improvement: We evaluate the workload performance of the proposed method compared to the baselines.

Experiment 2-1: Impact of insert operations: We examine how the number of insert operations affects the optimization cost for the proposed method and the baselines.

Table 1: Experimental environment

Machine	MacBook Pro (16-inch, 2024)
OS	macOS Sequoia 15.4.1(24E263)
CPU	Apple M4 Pro (14-core CPU, 20-core GPU)
Memory	48GB
ILP solver	Gurobi 12.0
Database system	PostgreSQL 17.4
Database	IMDb
Database size	7085MB

Experiment 2-2: Impact of storage budget: We analyze how the storage budget size affects the total view utility and the optimization time by comparing the proposed method and the baselines.

Experiment 3: Effectiveness of local search: To evaluate the effectiveness of local search, we compare the performance of our method with and without local search applied to three different initial solutions.

6.1 Setup

We describe benchmark and environment on our experimental study.

6.1.1 Benchmark. We conduct the experiments using the Redbench⁴, which is a recently published benchmark extended from the Join Ordering Benchmark (JOB) [17] by introducing query frequency. The benchmark runs on the IMDb database, consisting of 21 tables. The query frequency is generated based on the real-world workload from Redset⁵, a dataset of query metadata published by Amazon Redshift, so Redbench is suitable for workload evaluations.

Redbench contains 1,073 JOB-based queries. Among these, there are 83 unique JOB queries⁶, corresponding to distinct query templates. Furthermore, the Redbench workload is divided into ten groups based on query repetitiveness, which is defined as the proportion of queries appeared in the query history. In addition, we manually added synthetic insert operations to the benchmark by randomly selecting base tables, because the Redbench workload does not support update operations.

6.1.2 Environment. Table 1 describes the environment we used in the experiments. In operating PostgreSQL, we have adopted the default configuration settings without modification. Regarding the convergence criteria for the optimization process, we apply the same approach to all methods: we repeat the iteration until no further improvement can be achieved in the objective function.

6.2 Experimental Results

6.2.1 Experiment 1: Workload Performance Improvement. This experiment evaluates the performance using the selected materialized views using Naive ILP, BIGSUBS, and the proposed method.

⁴<https://github.com/utndatascience/redbench>

⁵<https://github.com/amazon-science/redset>

⁶While the complete JOB workload comprises 113 queries, Redbench selects a subset of 83 queries that involve between 6 and 11 joined tables to construct its benchmark workload.

Assuming a practical usage scenario, we first applied the view selection optimization using the original 113 JOB queries, which serve as the query templates for the Redbench workload. We then evaluated the workload performance using the 1,073 JOB-based queries that constitute the Redbench workload. Specifically, we measure the workload execution time using the selected materialized views, the optimization time for view selection, the total view utility of the selected materialized views, and the time taken to materialize the selected views. We generate 1,000 synthetic insert operations. We set the default size of the storage budget to 50 MB, which is approximately 10% of the total size required to materialize all queries of the original JOB benchmark.

Table 2 presents a detailed comparison of workload execution times, and corresponding speedups relative to the case without materialized views (w/o MV). The results are segmented by query repetition buckets indicating repetition ratio. Overall, all the proposed methods (topk-*F*, topk-*U*, and topk-*E*) demonstrate substantial improvements in the workload execution time compared to BIGSUBS and w/o MV method. In particular, the proposed method (topk-*F*), which selects the initial materialized views based on sub-query frequency, is 2.26 times faster than BIGSUBS and 4.59 times faster than w/o MV method, while being only 12.5% worse than the naive ILP, which identifies the optimal solutions. In the three query repetition buckets with the lowest query repetition (0%–30%), the proposed method (topk-*F*) achieves a higher speedup than the Naive ILP. This result suggests that the initial solution selection strategy based on query frequency tends to favor sub-queries that are shared across the entire workload, allowing it to be effective even when the query repetition rate is low.

Table 3 provides a comprehensive overview of performance metrics, total workload rerun (WR) time (optimization time for view selection + MV generation time + workload execution time), along with individual breakdowns of optimization time, and MV generation time. We also report the workload execution time improvement over w/o MV method and the utility of selected views. An interesting observation is that the proposed (topk-*F*) achieves the fastest rerun time (371.00 sec). This indicates that when considering the end-to-end process, which includes all the overhead of MV selection and generation alongside the actual workload execution, the proposed (topk-*F*) offers the most effective solution among the evaluated methods.

The proposed methods, particularly topk-*F* and topk-*U*, demonstrate a good balance between the effectiveness and efficiency of the view selection. They achieve near-optimal view utility with substantially lower overhead than ILP, making them promising candidates for practical use cases. In contrast, while the naive ILP can identify the materialized views with the highest utility, its significant optimization overhead makes it less practical for minimizing the workload rerun time. BIGSUBS, despite its efficient MV selection, suffers from lower view utility, which compromises its overall effectiveness in accelerating the workload.

We also investigate the Pearson correlation coefficient between the actual improvement in workload execution time using materialized views and the estimated total utility of the selected views. We find a strong correlation coefficient of 0.920. This confirms that the estimated utility of the selected materialized views effectively contributes to actual improvements in workload execution time.

Table 2: Workload execution time (sec) and speedup (%) relative to the case without using materialized views (w/o MV) across methods for different query repetition buckets indicating repetition ratio. The storage budget ($B_{\max}$) is set to 50 MB. The bold indicates the highest speedup in each query repetition bucket, and the underline indicates the second highest.

Query repetition buckets	w/o MV	Naive ILP		BIGSUBS		Proposed (topk-F)		Proposed (topk-U)		Proposed (topk-E)	
	Time	Time	Speedup	Time	Speedup	Time	Speedup	Time	Speedup	Time	Speedup
0%–10%	55.75	9.68	<u>82.64</u>	17.55	68.52	7.69	86.20	12.99	76.70	21.72	61.04
10%–20%	21.43	1.46	<u>93.19</u>	6.41	70.08	1.03	95.19	7.42	65.36	11.89	44.48
20%–30%	89.60	15.54	<u>82.66</u>	43.84	51.07	11.55	87.11	49.86	44.35	51.43	42.61
30%–40%	71.19	7.03	<u>90.12</u>	14.59	79.50	6.57	<u>90.77</u>	5.64	92.08	17.74	75.09
40%–50%	144.91	21.70	85.03	64.25	55.66	26.30	81.85	21.70	<u>85.02</u>	56.75	60.84
50%–60%	4.32	0.55	87.33	0.51	88.16	0.31	<u>92.77</u>	0.30	93.07	4.00	7.29
60%–70%	85.12	18.86	77.84	40.21	52.76	34.01	60.04	29.70	<u>65.10</u>	33.86	60.22
70%–80%	107.16	12.27	88.55	45.70	57.35	15.93	85.13	15.80	<u>85.25</u>	55.85	47.88
80%–90%	70.19	35.92	<u>48.82</u>	76.59	-9.12	39.69	43.44	32.13	54.22	47.92	31.73
90%–100%	226.08	43.92	80.57	121.80	46.13	47.61	<u>78.94</u>	202.60	10.39	186.68	17.43
TOTAL	875.73	166.93	80.94	431.45	50.73	190.71	<u>78.22</u>	378.14	56.82	487.84	44.29

Table 3: Comparison of total workload rerun (WR) time (optimization time for view selection + MV generation time + workload execution time), including workload execution time improvement over 'w/o MV', and utility of selected views. All times are reported in seconds.

Method	WR Time (s)	Opt. Time (s)	MV Gen. Time (s)	Improvement vs w/o MV (s)	MV Utility (K)
w/o MV	875.73	0	0	0.00	0
Naive ILP	440.68	79.75	194	708.80	3374
BIGSUBS	508.18	0.73	76	444.28	1855
Prop. (topk-F)	371.00	1.29	179	685.02	3272
Prop. (topk-U)	488.90	0.76	110	497.59	3344
Prop. (topk-E)	601.59	0.75	113	387.89	2707

6.2.2 Experiment 2-1: Impact of the number of insert operations. We conducted this experiment to evaluate the effectiveness of the proposed method with respect to the number of insert operations in the workload. We compared three variants of our method using different initial solutions against the baselines. We manually generated synthetic insert operations and changed the number of inserted records from 0 to 2,000 in increments of 100.

Figure 3 shows that the proposed methods, topk-F and topk-U achieve high utility comparable to the naive ILP regardless of the number of insert operations. The topk-E method, which balances utility and storage requirements by selecting initial sub-queries based on their utility per unit of storage, is hypothesized to converge to a local optimum. In addition, the total utility achieved by BIGSUBS is mostly the lowest and it does not exhibit a consistent decline but rather fluctuates. This instability in the total utility is attributed to its probabilistic approach, as noted in prior work [14].

6.2.3 Experiment 2-2: Impact of storage budget. This experiment was conducted to compare the total utility and the optimization time between the proposed methods and the baselines. The number of insert operations was fixed at 1,000, and the storage budget size $B_{\max}$ was changed across 5 MB, 50 MB, and 500 MB.

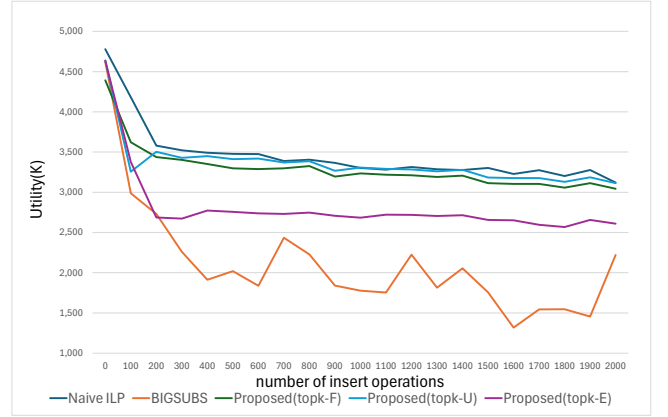


Figure 3: Total view utility with respect to the number of insert operations.

Overall, the results verify that the proposed methods achieve a good balance between the total utility and the optimization time across all values of $B_{\max}$. They achieve comparable total utility to the naive ILP and high efficiency for the optimization time. Regarding the total utility, Figure 4a shows the proposed methods consistently yield the higher total utility across all values of $B_{\max}$ compared to BIGSUBS. This observation verifies the lower accuracy inherent in the BIGSUBS approach. In addition, we can observe a natural phenomenon that the total utility increases as the storage budget size increases and the total utility tends to converge. That is, the selected views are getting close to materializing all queries in the workload.

Regarding the optimization time, Figure 4b shows that both BIGSUBS and the proposed methods significantly outperform the naive ILP, achieving substantial reductions in optimization time. Among the proposed methods, topk-F and topk-E exhibit relatively longer optimization times. We conjecture that their initial solutions are largely different from the optimal ones, which require more iterations. In contrast, topk-U is presumed to select initial solutions

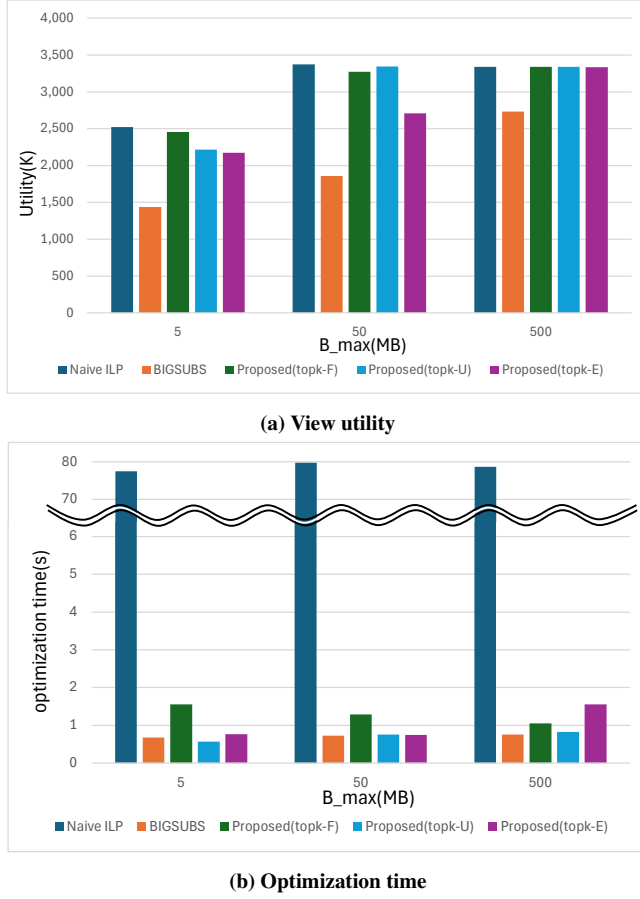


Figure 4: Comparison of total view utility and optimization time with respect to B_{max} (5 MB, 50 MB, and 500 MB).

closer to the optimal ones, thereby reducing the number of iterations and overall optimization time. Also, BIGSUBS is efficient in optimization time, but the total utility is worse than the proposed methods.

6.2.4 Experiment 3: Effectiveness of Local Search. This ablation experiment was conducted to evaluate the effectiveness of the local search in the proposed methods. We compared the optimization time and the total utility obtained by our methods with and without the local search. That is, the initial solutions are used as selected views for the case without using the local search. The number of insert queries was set to 1,000.

Table 4 shows that local search provides a substantial benefit in increasing total utility for topk-F and topk-E, with only a marginal additional time. As an example of topk-F, local search improves total utility by a factor of 2.1, incurring only 1.18 seconds of additional optimization time: it is significantly smaller than the whole workload execution time of approximately 100 seconds (see Table 2). Note, in the case of topk-U, the total utility remains unchanged, indicating that the initial solutions are better than or equal to the ILP-refined solutions, and thus the iteration terminates after the first round.

Table 4: Comparison of total view utility and optimization time with and without local search (B_{max} : 50MB). *diff* indicates the difference between the proposed method with and without local search (proposed - w/o local search).

Method	Utility (K)	Optimization Time (sec)
proposed (topk-F)	3270	1.29
w/o local search (topk-F)	1570	0.11
<i>diff</i>	+1700	+1.18
proposed (topk-U)	3340	0.76
w/o local search (topk-U)	3340	0.28
<i>diff</i>	+0	+0.48
proposed (topk-E)	2710	0.75
w/o local search (topk-E)	2280	0.02
<i>diff</i>	+430	+0.73

7 Related Work

The selection of materialized views has been extensively studied, with comprehensive surveys [11, 18] providing overviews of a variety of strategies and their trade-offs, including view selection, query rewriting, and answering queries using views. A recent survey [29] reviews the techniques for both AI4DB and DB4AI, including database configuration tuning, optimizer, and index/view advisor. The techniques for view selection are categorized into heuristic-based approaches [6, 8, 9, 12, 13, 23] or optimization-based approaches that formulate the problem using integer linear programming (ILP) [5, 14, 19, 27].

7.1 Heuristic-based approaches

Classical techniques for view selection in data warehouses [8, 9, 23] clarify the theoretical aspect of view selection and propose greedy approaches. These techniques rely on heuristics to search the space of candidate views and are effective for small workloads, but they suffer from limited scalability due to their local search nature and reliance on simplifying assumptions.

7.2 ILP-based approaches

ILP-based approaches have emerged as a principled strategy for optimizing view selection under complex constraints. These methods formulate the problem as an integer linear program where the objective is to maximize overall benefit or minimize total cost subject to storage and maintenance budgets. Papadomanolakis and Ailamaki [19] proposed one of the first ILP-based frameworks for database physical design, originally targeting index selection but extensible to materialized view selection. PigReuse [5] applies ILP-based selection over AND/OR DAGs for Pig Latin scripts, enabling the reuse of common subplans. A prominent ILP-based system, BIGSUBS [14], reformulates the view selection problem as a labeling task over a bipartite graph. It decomposes the large ILP into smaller subproblems and iteratively updates the set of materialized sub-queries. While scalable to large workloads, BIGSUBS relies on probabilistic heuristics and does not guarantee convergence to a global optimum. Wide-deep [27] extends BIGSUBS by introducing a neural model

to estimate view utility and uses deep reinforcement learning to guide view selection. However, this approach introduces additional complexity and may lead to instability in training and convergence. In contrast, our work is the first work that employs local search, and it achieves high fast optimization speed and high quality of selected views.

7.3 Multi-query optimization

Multi-query optimization (MQO) is a related but distinct problem that focuses on identifying opportunities to share computation across a batch of concurrent queries. MQO techniques typically use temporary views, which are materialized only during the execution of the batch. Roy et al. [21] extend the Volcano optimizer with AND/OR DAGs to identify and reuse common subplans across queries, achieving significant cost savings. Koch and Schlegel [15] propose an algebraic approach that merges multiple queries into a unified representation, allowing efficient factoring of shared computations beyond pairwise reuse. More recently, Anneser et al. [4] introduce AutoSteer, a learning-based framework that tunes optimizer parameters to improve performance across batches of similar queries, achieving up to 40% speedup on engines like PostgreSQL and SparkSQL.

8 Conclusion

We proposed a new method that incorporates incremental view maintenance cost directly into the optimization objective and applies local search to efficiently explore the solution space. In order to apply local search to the view selection problem, we define neighboring solutions using sub-query containment, and select initial solutions based on sub-query frequency, utility, or utility per storage unit. We confirmed the practical effectiveness of our view selection method using a recent benchmark, RedBench.

Future work includes the following directions:

- (1) Evaluating the effectiveness of materialized views on more generalized workloads
- (2) Utilizing PostgreSQL's incremental view maintenance functionality⁷
- (3) Applying the wvlet framework for extracting logical plans, generating view definitions, and rewriting queries⁸

Since the JOB workload used in the experiments does not include common query patterns such as nested queries, it is necessary to investigate how the effectiveness of the proposed method and BIGSUBS changes when applied to more generalized workloads. Moreover, by implementing PostgreSQL's incremental view maintenance feature, we plan to evaluate the update cost during workload execution. In addition, since wvlet, a flow-based query language, allows direct extraction of logical plans, we aim to explore the feasibility of integrating our method with this framework.

9 GenAI Usage Disclosure

We gratefully acknowledge the use of generative AI tools in the preparation of this manuscript. Specifically, we used [ChatGPT-4o] for the following purposes:

- Translation Assistance: The AI tool was employed for English proofreading and for translating our native language text into English. The authors thoroughly reviewed all translated content and made revisions to ensure accuracy and clarity.
- LaTeX Code Generation for Tables: Based on data collected and organized by the authors, the AI tool was used to generate LaTeX code for formatting tables. The authors verified the correctness of the generated code and ensured appropriate data presentation.
- Programming Support: During the development of the programs used in the experiments, the AI tool assisted in debugging and suggesting modifications. All code modifications proposed by the AI were reviewed, tested, and validated by the authors.

The authors take full responsibility for the final content and integrity of this research.

References

- [1] E. H. L. Aarts and J. K. Lenstra. 1997. *Local Search in Combinatorial Optimization*. Princeton University Press.
- [2] Supun Abeyasinghe, Qiyang He, and Tiark Rompf. 2022. Efficient incrementalization of correlated nested aggregate queries using relative partial aggregate indexes (RPAI). In *Proceedings of SIGMOD*. 136–149.
- [3] Yanif Ahmad and Christoph Koch. 2009. DBToaster: a SQL compiler for high-performance delta processing in main-memory databases. *Proc. VLDB Endow.* 2, 2 (2009), 1566–1569.
- [4] Christoph Anneser, Nesime Tatbul, David Cohen, Zhenggang Xu, Prithviraj Pandian, Nikolay Laptev, and Ryan Marcus. 2023. AutoSteer: Learned Query Optimization for Any SQL Database. *PVLDB* 16, 12 (2023), 3515–3527.
- [5] J. Camacho-Rodríguez, D. Colazzo, M. Herschel, I. Manolescu, and S. R. Chowdhury. 2016. Reuse-Based Optimization for Pig Latin. In *Proceedings of CIKM*. 2215–2220.
- [6] Tansel Dokeroglu, Murat Ali Bayir, and Ahmet Cosar. 2015. Robust heuristic algorithms for exploiting the common tasks of relational cloud database queries. *Applied Soft Computing* 30 (2015), 72–82.
- [7] Ashish Gupta, Inderpal Singh Mumick, and V. S. Subrahmanian. 1993. Maintaining views incrementally. In *Proceedings of SIGMOD*.
- [8] H. Gupta. 1997. Selection of Views to Materialize in a Data Warehouse. In *Proceedings of ICDT*. 98–112.
- [9] H. Gupta and I. S. Mumick. 2005. Selection of views to materialize in a data warehouse. *IEEE Transactions on Knowledge and Data Engineering* 17, 1 (2005), 24–43.
- [10] Alon Y Halevy. 2001. Answering queries using views: A survey. *The VLDB Journal* 10 (2001), 270–294.
- [11] A. Y. Halevy. 2001. Answering queries using views: A survey. *The VLDB Journal* 10, 4 (2001), 270–294.
- [12] Venky Harinarayan, Anand Rajaraman, and Jeffrey D Ullman. 1996. Implementing data cubes efficiently. *ACM Sigmod Record* 25, 2 (1996), 205–216.
- [13] V. Harinarayan, A. Rajaraman, and J. D. Ullman. 1996. Implementing Data Cubes Efficiently. In *Proceedings of SIGMOD*. 205–216.
- [14] Alekh Jindal, Konstantinos Karanasos, Sriram Rao, and Hiren Patel. 2018. Selecting subexpressions to materialize at datacenter scale. *Proc. VLDB Endow.* 11, 7 (2018).
- [15] Christoph Koch and Thomas Schlegel. 2004. Multi-Query Optimization Revisited: A Full-Query Algebraic Method. In *Proceedings of VLDB*. 539–550.
- [16] Hai Lan, Zhifeng Bao, and Yuwei Peng. 2021. A survey on advancing the DBMS query optimizer: Cardinality estimation, cost model, and plan enumeration. *Data Science and Engineering* 6 (2021), 86–101.
- [17] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2015. How good are query optimizers, really? *Proceedings of the VLDB Endowment* 9, 3 (2015), 204–215.
- [18] I. Mami and Z. Bellahsene. 2012. A Survey of View Selection Methods. *SIGMOD Record* 41, 1 (2012), 20–29.
- [19] Stratos Papadomanolakis and Anastassia Ailamaki. 2007. An Integer Linear Programming Approach to Database Design. In *Proceedings of ICDE Workshop*. 442–449.
- [20] Raghu Ramakrishnan, Baskar Sridharan, John R Douceur, Pavan Kasturi, Balaji Krishnamachari-Sampath, Karthick Krishnamoorthy, Peng Li, Mitica Manu, Spiro Michaylov, Rogério Ramos, et al. 2017. Azure data lake store: a hyperscale distributed file service for big data analytics. In *Proceedings of SIGMOD*. 51–63.

⁷https://wiki.postgresql.org/wiki/Incremental_View_Maintenance

⁸<https://github.com/wvlet/wvlet>

- [21] Prasan Roy, S. Seshadri, and S. Sudarshan. 2000. Efficient and Extensible Algorithms for Multi-Query Optimization. In *Proceedings of SIGMOD*. 249–260.
- [22] Yasin N Silva, Paul-Ake Larson, and Jingren Zhou. 2012. Exploiting common subexpressions for cloud query processing. In *Proceedings of ICDE*. IEEE, 1337–1348.
- [23] D. Theodoratos and T. K. Sellis. 1997. Data Warehouse Configuration. In *Proceedings of VLDB*. 126–135.
- [24] Zuozhi Wang, Kai Zeng, Botong Huang, Wei Chen, Xiaozong Cui, Bo Wang, Ji Liu, Liya Fan, Dachuan Qu, Zhenyu Hou, Tao Guan, Chen Li, and Jingren Zhou. 2023. Tempura: a general cost-based optimizer framework for incremental data processing (Journal Version). *The VLDB Journal* 32, 6 (2023), 1315–1342.
- [25] Ziniu Wu, Parimarjan Negi, Mohammad Alizadeh, Tim Kraska, and Samuel Madden. 2023. FactorJoin: A New Cardinality Estimation Framework for Join Queries. *Proceedings of SIGMOD* 1, 1 (2023).
- [26] Zongheng Yang, Amog Kamsetty, Sifei Luan, Eric Liang, Yan Duan, Xi Chen, and Ion Stoica. 2020. NeuroCard: one cardinality estimator for all tables. *Proc. VLDB Endow.* 14, 1 (2020), 61–73.
- [27] Haitao Yuan, Guoliang Li, Ling Feng, Ji Sun, and Yue Han. 2020. Automatic view generation with deep learning and reinforcement learning. In *Proceedings of ICDE*. IEEE, 1501–1512.
- [28] Jingren Zhou, Per-Ake Larson, Johann-Christoph Freytag, and Wolfgang Lehner. 2007. Efficient exploitation of similar subexpressions for query processing. In *Proceedings of SIGMOD*. 533–544.
- [29] Xuanhe Zhou, Chengliang Chai, Guoliang Li, and JI SUN. 2020. Database Meets Artificial Intelligence: A Survey. *IEEE Transactions on Knowledge and Data Engineering* 34, 3 (2020).